Harbor Volume — Paper 2 of 4

The Single-Writer Kernel:

A Local Transactional Reference Monitor for Agent Swarms

Erich Owens

Curiositech LLC

engineering@portdaddy.dev

June 2026
Version 1.0 (Harbor Volume)

Abstract

A swarm of coding agents sharing one machine collides over the same scarce things: ports, files, locks, and the truth about who did what. The instinct, trained on a decade of distributed-systems literature, is to reach for consensus — replicate the state, run an agreement protocol. Port Daddy's kernel makes the opposite, and we argue correct, move: it collapses the entire problem onto a **single writer over a single local SQLite file in write-ahead-log mode**, and lets the operating system's file lock serialize every mutation. There is one decider, so there is no agreement to reach.

This paper presents that kernel (layer **L0** of the Harbor stack) and the *built* half of the coordination substrate above it (layer **L1**: tube, pheromones, commitments, the cryptographic `auth-chain`, the Arbiter) as a **single-writer transactional reference monitor** in the Lampson–Anderson sense: a small, tamper-evident mediator of every security-relevant operation, realized locally. We state the kernel's invariants as theorems — mutual exclusion, idempotence, the consistency model (serializable transactions, linearizable claim history), oracle-bound closure — and we are deliberate about where the promises stop. In particular we *split* durability by fault class: a returned-success write survives a *process* crash but is *not guaranteed* against power loss under the shipped pragma set; and we correct a widespread over-claim, showing that the Arbiter is a post-commit *detector*, not a pre-commit regimenter, with real regimentation credited to the `PRIMARY KEY` constraint and the boot gate. The kernel is solid where it is local and provisional exactly where the economy (Paper 4) will need it to become cryptographic and continuous — and we say so in the same words throughout.

Keywords: reference monitor, single-writer transactions, SQLite WAL, durability by fault class, linearizability, deontic logic, commitments, stigmergy, local-first coordination, multi-agent systems

Reading time: approximately 45 minutes end-to-end; the Reader's Map (§2) routes you to the 15-minute and one-section paths. This is Paper 2 of the four-paper Harbor Volume. Paper 1, The Legible Swarm*, is the operator-facing wedge and the natural place to start; Paper 3,* From

Spawn to Person*, builds the identity-and-reputation bridge on top of this kernel; Paper 4,* The Harbor Economy*, carries the cross-machine market — including the "Anchor Protocol proper" (cross-harbor capability transfer), which despite the old name is L3 and lives there, not here. Any paper can be read first, but every other paper cites this one.*

Contents

1 Where this paper sits, and what it promises

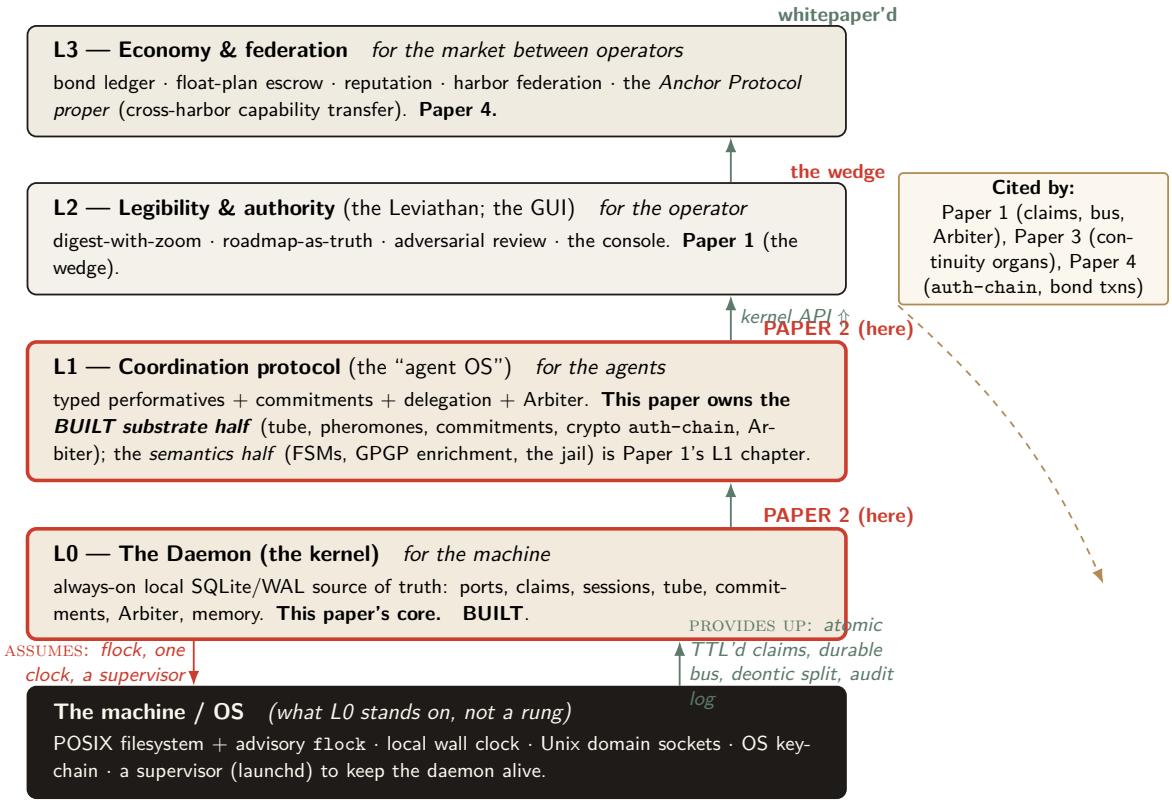


Figure 1: The Harbor Volume stack-map. Each rung serves a different *whom* (machine, agents, operator, market). **This paper (Paper 2) owns the two lowest rungs**: all of L0 (the kernel) and the *BUILT substrate half* of L1 (tube, pheromones, commitments, the cryptographic `auth-chain`, the Arbiter). The L1 *semantics* (protocol FSMs, GPGP-enriched commitments, the jail) and all of L2 belong to Paper 1; the *Anchor Protocol proper* (cross-harbor capability transfer) is L3 and belongs to Paper 4. Cinnabar shows what L0 *assumes* from the machine; patina shows what the kernel *provides* upward. Papers 1, 3, and 4 all cite this one — it is the load-bearing substrate the rest of the volume stands on.

Every paper in the Harbor Volume opens with the same map (Figure 1) so the volume reads as one thing rather than four overlapping drafts. The map is a ladder. Each rung serves a different *whom*: the **machine** (L0, the kernel — this paper's core), the **agents** (L1, the coordination protocol — this paper owns its *built* substrate), the **operator** (L2, legibility — Paper 1), and the **market between operators** (L3, economy — Paper 4). You climb the ladder in order, and you never buy a rung before you need it.

This paper is the load-bearing floor. The other three cite it: Paper 1 reads the kernel's claims, bus, and Arbiter to render the swarm; Paper 3 leans the entire notion of a *person* on the kernel's continuity organs; Paper 4's bond ledger is, mechanically, another set of tables under the same single writer, and its cross-harbor capability transfer rides the kernel's cryptographic `auth-chain`. If this floor is unsound, everything above it inherits the fault. So the discipline of this paper is to state each promise precisely and, where the promise has an edge, to draw the edge in ink.

1.1 The one-sentence thesis

> *Port Daddy's kernel is a single-writer transactional reference monitor over a local SQLite/WAL file — the one process that mediates contended resources, and the one place where "true" is decided rather than asserted.*

That is the whole claim, and it has two halves, which are the kernel's entire job; everything else is convenience.

1. **Durability (with an edge):** a write that returned success is durable across the death of any agent — and, we will be careful, across the death of the *daemon process*, though *not* unconditionally across power loss.
2. **Mediation (with a correction):** a state the kernel *regiments* cannot come to exist; a state the kernel merely *enforces* is detected and compensated within a bounded window. These are different guarantees and we will never conflate them.

1.2 Why "reference monitor," and why *not* consensus

The right mental model is the **reference monitor** of Anderson and Lampson [1, 2]: a mediator that is *always invoked* (complete mediation), *tamper-evident*, and *small enough to verify*. The kernel instantiates this not as an abstract security kernel but as a concrete local daemon over one SQLite file. Complete mediation is achieved *by construction*: because every mutation routes through one writer holding one file lock, the operating system's serialization *is* the mediation primitive — Saltzer and Schroeder's "economy of mechanism" [3] done with one moving part instead of N hand-rolled critical sections.

Key idea. The swarm *looks* like a distributed-systems problem — many agents, contention, failover — so the trained reflex is Raft, Paxos, CRDTs. The kernel's defining move is to refuse that problem. One writer, one machine, one file: there is nothing to *agree* on, so the consensus impossibility of Fischer, Lynch, and Paterson [11] does not bite. It is sidestepped, not solved. Distribution is an L3 concern (Paper 4); pushing Raft down into L0 would be the wrong layer.

This is the single most important architectural decision a local-first coordination layer can make, and the field is littered with systems that got it wrong by manufacturing a consensus problem they did not have. We will state the formal payoff of the choice as a theorem in §11: a single writer over WAL gives *serializable* transactions, and one decider gives *linearizable* external behavior for claims and locks — the property that lets the protocol layer treat "who holds this" as ground truth without any agreement protocol at all.

1.3 An honest-label key, and why the labels are load-bearing

This paper inherits the volume's honest-label discipline (ADR-0045). Every claim carries one of four labels — and two refinements this paper introduces because a single label cannot honestly carry a claim that spans fault classes.

Table 1: The honest-label key, used in every paper of the volume. The last two rows are this paper's refinements (see §5 and §8).

Label	Meaning
BUILT	Shipped, exercised, and true under the production runtime.
BUILT-WEAK	Shipped but partial: works under a narrower condition than the prose might invite, or detects rather than prevents.
DESIGNED	Specified (an ADR exists) but not in the shipped tree.
VISION	Named as a direction; no specification yet.
I1a / I1b	Durability *split by fault class*: I1a (process-crash) is **BUILT**; I1b (power-loss) is **NOT GUARANTEED** under the shipped pragmas. A claim that conflates fault classes cannot carry one label.
regimented / enforced	Prohibition split by interception point: *regimented* = rejected pre-commit, truly unreachable (PK constraint / boot gate); *enforced* = detected post-commit, halted/compensated within a bounded window (the Arbiter). "Physically unreachable" is reserved for the regimented set.

Pitfall. The labels are not decoration. The two most consequential corrections in this paper — the durability split and the regimentation/detection split — are exactly the places where a single optimistic label would let the kernel's foundational promise be read as stronger than the code delivers. When a pitch says "the file survives every agent's death," a reader hears "power-loss durable." It is not, by default. Saying so is the difference between a kernel paper and marketing.

2 Reader's Map

Table 2: Reader's Map. Find your row; read those sections first.

If you are...	...read first	...load-bearing artifact
short on time (15 min)	§1 (thesis + stack map), §5 (durability split), §8 (Arbiter correction)	Figs. 1, 4, 8
a systems engineer	§4–§11 (substrate, single-writer, the theorems)	Thm. 11.1; Figs. 3, 11
a security reviewer	§8 (deontic split, Arbiter), §13 (threat model)	Figs. 8, 7; Table 4
a protocol/agent author	§6 (claims), §7 (the bus), §14 (the API contract)	Figs. 5, 6; §14
here for the economy	§9 (continuity organs), then jump to Paper 3/4	Fig. 10 (the C7 sentence)
an instructor	every **Exercises** block; the open problems OP-1 – OP-9 (§15)	the starred exercises

Reading order within the volume. Paper 1 (the wedge) is the gentlest on-ramp; this paper (the substrate) is the formal floor; Paper 3 (the person) and Paper 4 (the market) build upward. If you only read one paper to decide whether the foundation is sound, read this one.

3 The kernel as seven organs

We organize the kernel into seven *organs*, each a contract the daemon must hold for the stack above to be coherent (Figure 2). The flat noun-list from the project's early notes — "ports, claims, sessions, tube, pheromones, commitments, Arbiter, memory" — is the symptom of treating the kernel as a database. It is not a database. It is a system of record plus a mediator, and naming the organs surfaces the connective tissue the noun-list hides.

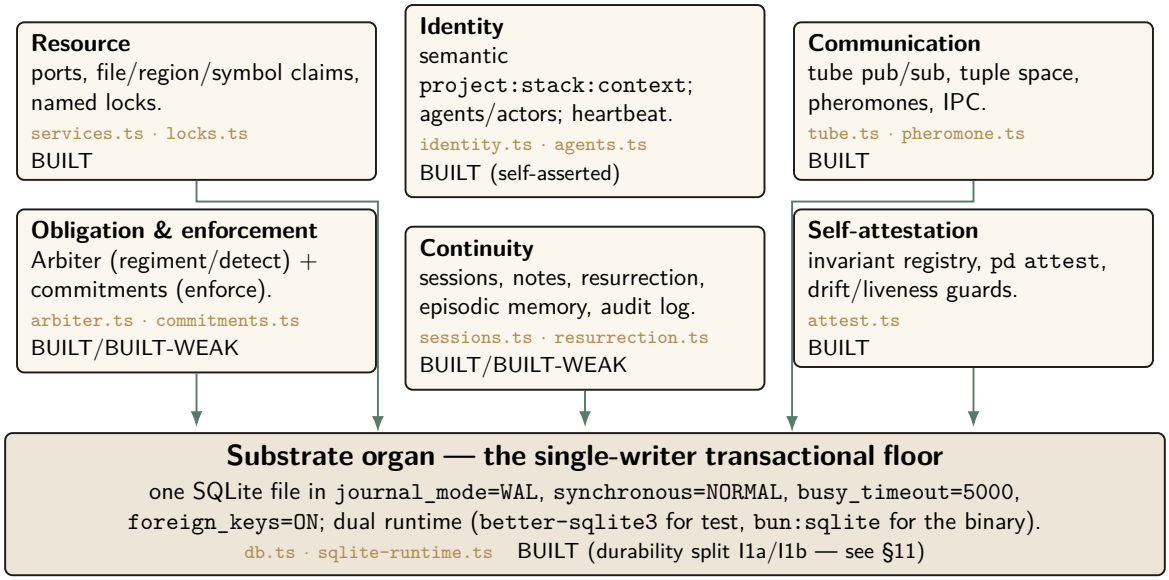


Figure 2: The kernel as *seven organs*, each a contract the daemon must hold for the stack above to be coherent. Six organs are, mechanically, just tables *with discipline* over a single shared file — the **substrate organ**, the single-writer transactional floor. The substrate's pragma set *is* the kernel's durability claim; everything else is convenience built on top of it. Honest labels (BUILT · BUILT-WEAK · DESIGNED · VISION) are per organ; note that the identity organ is BUILT but *self-asserted*, the gap the Anchor Protocol proper (L3) eventually closes.

The **substrate organ** is the floor; the other six are, mechanically, tables *with discipline* over the same file. We treat the substrate and the two organs that carry the paper's sharpest corrections (resource/exclusion and obligation/enforcement) in full, and the rest with the depth their novelty warrants.

Exercises.

(check) Which of the seven organs would still be meaningful if the daemon ran over an in-memory database with no disk at all? Which become vacuous?

(trace) Pick any one organ and name the single SQLite table that is its "home" table. Then name one invariant that organ owes the layer above it.

(open, ★) The seven-organ decomposition is a *choice*. Is there a more natural cut — e.g. five organs, or nine? Argue for an alternative and show what it makes easier to reason about and what it hides.

4 The substrate organ: one writer, one file

The substrate is the bedrock everything else is just a table in. Its job is to make two promises — durability and serialized mutation — and to make them honestly. We cover the single-writer

discipline here and durability in §5, because durability is where the most consequential correction lives.

4.1 The single-writer discipline

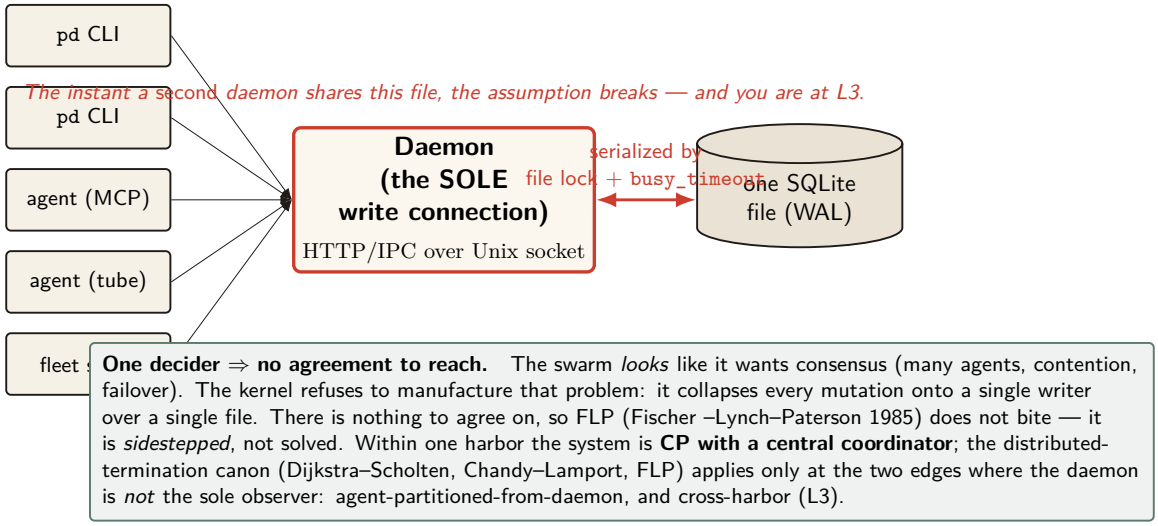


Figure 3: The single-writer serialization, the cleanest thing about the layer. Dozens of CLI invocations and agents funnel through *one* writer — the daemon, the sole holder of a read–write connection — and SQLite's OS-level file lock plus `busy_timeout` serialize all mutations. Because there is exactly one decider, there is no agreement to reach: the consensus impossibility results that dominate distributed-systems design are *deliberately not* L0 problems. This is not a gap; it is the layer's defining discipline. The boundary is exact: the moment a second daemon touches the same truth, you have a distributed problem — and that is L3, treated in Paper 4, not here.

The kernel's concurrency story (Figure 3) is: dozens of `pd` invocations and agents funnel through one writer — the daemon, the sole holder of a read–write connection to the database file (`lib/db.ts`, the module that opens and configures the SQLite handle) — and SQLite's operating-system file lock plus `busy_timeout=5000` serialize all mutations. We must be precise about *what* serializes writers, because there are two stories and only one is true at a time.

Definition 4.1 (Single-writer discipline)**.** All state-mutating operations route through a single daemon process holding one SQLite write connection. Concurrency among the many clients (CLI invocations, agents over MCP or tube, fleet sorties) is resolved by the daemon's request queue; the SQLite file lock is the *backstop* that preserves correctness if the discipline is ever violated by a second writer (for example a stray direct-CLI write), not the primary serialization mechanism.

Key idea. "Single-writer" is an architectural *discipline* (everything goes through the daemon), backstopped — not replaced — by SQLite's file lock. This distinction matters: SQLite permits multiple writer connections (it serializes them, it does not reject them), so the property holds because of where writes are routed, not because the database forbids a second writer. The same risk that motivates the kernel's binary-drift guards (a second *copy* of the daemon running) is the risk that a second writer appears.

4.2 The pragma set is the durability claim

The substrate's configuration is short and load-bearing. In WAL mode (`journal_mode=WAL`) with `synchronous=NORMAL`, `wal_autocheckpoint=200`, `busy_timeout=5000`, `foreign_keys=ON`, and a clean-shutdown `wal_checkpoint(TRUNCATE)`, the kernel inherits exactly the crash semantics that pragma set defines. The claim "a write that returned success survives a crash" reduces *entirely* to what WAL with `synchronous=NORMAL` guarantees — which is the subject of §5, and which is not what an unguarded reading assumes.

4.3 Schema by union, not by sequencer

There is no migration sequencer. Every module self-initializes its tables with `CREATE TABLE IF NOT EXISTS` plus lazy `ALTER TABLE` migrations; the schema is the *union* of what each `createFoo(db)` factory declares, in whatever order modules load. This is elegant for greenfield — any organ can create its own tables independently — and a latent hazard for renames, backfills, and multi-column migrations, for which there is no down-migration and no version table (open problem OP-7, §15).

4.4 Path and ownership invariants

The database resolves to a single canonical path with `chmod 0o600` (the file historically carried the Ed25519 signing key in plaintext; that secret is mid-migration to the OS keychain via `lib/keychain.ts`, see §13). A fail-closed guard, `assertNotProdInTest` — the analogue of Rails' `ProtectedEnvironment` — refuses to open the live registry from a test context. This is Saltzer–Schroeder fail-safe-defaults applied at the one place a test could corrupt production truth.

Exercises.

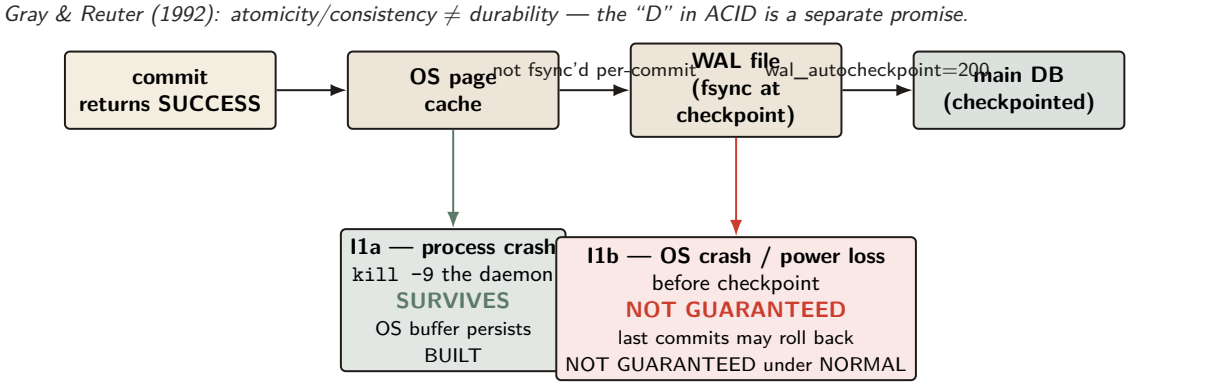
(check) The schema-by-union design lets any module create its tables in any order. Give a concrete two-module example where a lazy `ALTER TABLE` rename would be unsafe under this design.

(trace) Follow a single `pd claim` from CLI to disk. At how many points does the single-writer discipline (Def. 4.1) hold by *routing* versus by the *file lock*?

(open, ★ OP-7) Can idempotent self-init be extended to safe renames/backfills/down-migrations while preserving "any module, any order"? Or is a version table unavoidable?

5 Durability, split by fault class

This is the most important correction in the paper, so it gets its own section. The kernel's foundational promise — "a write that returned success survives" — is true, but only for the fault class a careful reader expects and not for the one the pitch most invites.



The trade made, stated honestly: `synchronous=NORMAL` fsyncs the WAL only at checkpoint, buying throughput at the cost of *power-loss* durability. The in-code comment (`db.ts:388`) "NORMAL is safe in WAL mode" conflates crash-*consistency* (the DB is never corrupt — true) with *durability* (committed work survives power loss — false). The remedy for I1b is `synchronous=FULL` or a checkpoint on the commit path; the kernel does not pay that cost by default, and the pitch must not imply it does.

Figure 4: Durability by fault class — the single most important correction to the kernel's foundational promise. The unqualified claim "a write that returned success survives a crash" is true for a *process* crash (I1a: the OS page cache persists past the dying daemon) but *not guaranteed* for an OS crash or power loss (I1b), because `synchronous=NORMAL` fsyncs the WAL only at checkpoint. This is exactly the Gray–Reuter atomicity-versus-durability distinction, and it is the one fault class a "the file survives every agent's death" pitch most invites the reader to over-assume. We split the invariant accordingly: I1a is BUILT; I1b is NOT GUARANTEED under the shipped pragma set.

5.1 Why one label is a lie here

Gray and Reuter [6] draw the canonical line between *atomicity/consistency* (a transaction is all-or-nothing and never leaves the store corrupt) and *durability* (the "D" in ACID: a committed transaction survives subsequent failures). Under SQLite WAL with `synchronous=NORMAL`, the WAL is *not* fsync'd on every commit — it is synced at checkpoint. Per SQLite's own documentation, in WAL mode `NORMAL` "commits might lose durability following a power loss or hard reset," though the database remains uncorrupted. So the honest statement splits in two.

Property 5.1 (Durability by fault class)**.** Let a write return success under the shipped pragma set.

- **I1a (process-crash durability).** If the *daemon process* dies (`kill -9`, a panic, an OOM), the write survives: the data is in the operating system's page cache, which outlives the process. **BUILT**.
- **I1b (power-loss durability).** If the *operating system* crashes or power is lost *before the next checkpoint*, the most recent committed writes may roll back. **NOT GUARANTEED** under `synchronous=NORMAL`; it would require `synchronous=FULL` or a checkpoint on the commit path.

Pitfall. The in-code comment justifying the pragma choice (`db.ts:388`) reads "NORMAL sync is safe in WAL mode — WAL already guarantees crash safety." This is the standard misconception: it conflates crash-*consistency* (true: the DB is never corrupt) with *durability* (false for power loss). The comment does not merely under-document the trade; it *contradicts* the Gray–Reuter citation the dossier rests I1 on. A kernel paper cannot ship the unqualified claim. We split it.

5.2 Why the trade is defensible — and where it stops being so

Trading power-loss durability for throughput is a reasonable default for a local-first developer tool: the failure (lose the last few hundred milliseconds of claims after a power cut) is benign and self-correcting — agents re-claim, heartbeats resume. What is *not* defensible is letting the pitch imply the stronger guarantee. The remedy for any caller who genuinely needs I1b — say, a future bond-ledger write at L3 where lost money is not benign — is to elevate that write path to `synchronous=FULL` or to force a checkpoint, and to pay the throughput cost knowingly. The kernel must expose that choice, not bury it.

5.3 A recovery story, not just a durability story

Durability is about what survives; recovery is about what happens next. On daemon restart with a dirty WAL, SQLite replays the log to a consistent state (I1a's mechanism). But the kernel-level questions are larger: who validates the audit chain on boot, and what becomes of a half-applied cross-organ operation (§12) after a crash? The hook exists — `pd attest`'s "refuse-to-serve on CRITICAL fail" (§10) — but boot-time WAL recovery plus integrity check plus chain verification should be a named invariant, and today it is closer to **BUILT-WEAK** than **BUILT**.

Exercises.

(check) Classify each fault as I1a-survivable or I1b-exposed: (a) `kill -9` of the daemon; (b) `sudo shutdown -r now`; (c) yanking the power cord; (d) an unhandled exception in a route handler.

(trace) A claim commits at t_0; the next `wal_autocheckpoint` fires at $t_0 + \delta$. Draw the window during which a power loss reverts the claim, and state how `wal_autocheckpoint=200` bounds δ in pages.

(open, ★ P1#5) Design a per-write-path durability selector: how would the kernel let a bond-ledger write opt into I1b while keeping claim writes fast? What is the cleanest API that does not re-introduce the conflation?

6 The resource organ: claims, locks, and fair exclusion

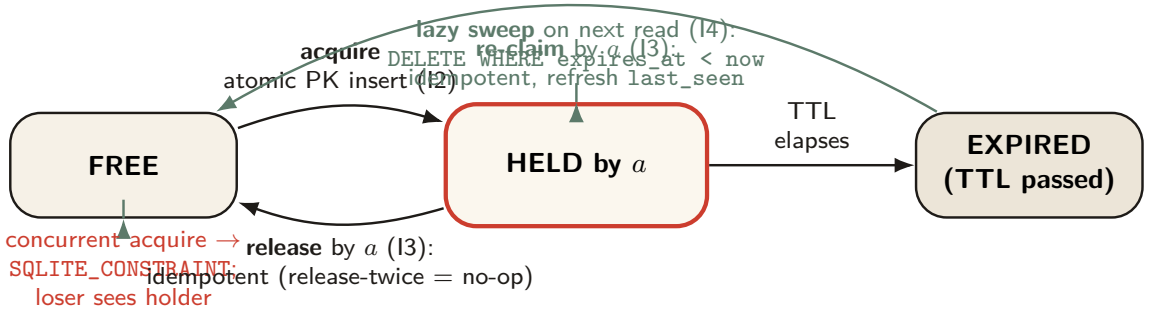
Ports are the namesake and the original sin. The resource organ (**`lib/services.ts`** for ports, **`lib/locks.ts`** for generic mutexes, the **`session_files`** table for edit-surface claims) is the exclusion primitive every higher layer's ownership reduces to.

6.1 Claim granularity is richer than "claims"

The early notes collapsed three primitives into one word. The kernel actually distinguishes **whole-file**, **line-range**, and **symbol-path** claims (the `session_files` columns `file_path`, `start_line`, `end_line`, `symbol`, `symbol_path`). This is the substrate for the project's "reserve

regions, not whole files" direction and for semantic-conflict prediction over a tree-sitter symbol index — but at L0 it is just exclusion at three granularities.

6.2 The claim lifecycle as a finite-state machine



★ **OP-1 (open problem):** there is *no* wait-list and *no* fairness. A fast agent that churns a hot file can starve a slow one indefinitely. The FSM has no QUEUED state; adding bounded-wait without a scheduler is open.

Figure 5: The claim/lock lifecycle as a finite-state machine, exhibiting the four properties on which L1 builds typed ownership. *Acquire* is an atomic `PRIMARY KEY` insert (I2: a concurrent acquirer loses cleanly to `SQLITE_CONSTRAINT` and is handed the holder). *Re-claim* and *release* are idempotent (I3): safe to retry, safe to double. Expiry is *lazy-swept* on the next read (I4), not on a timer. The conspicuous missing state is QUEUED: there is no fairness primitive, so a hot resource can starve a slow agent — open problem OP-1, which asks whether bounded-wait can be added without dragging a scheduler down into the kernel.

Figure 5 states the four properties L1 builds typed ownership on, as a state machine. We give the central one as a theorem.

Theorem 6.1 (Atomic mutual exclusion)**.** At most one holder exists per resource key (port | file-region | named lock) at any instant. The acquire path is a bare `INSERT` against a `PRIMARY KEY`; a concurrent acquirer's insert fails with `SQLITE_CONSTRAINT`, and the loser is handed the current holder.

Proof sketch. Acquire is a single atomic `INSERT` on the resource key, which is the table's primary key. SQLite guarantees at most one row per primary-key value; the second concurrent insert violates the constraint and is rejected atomically. Because the operation is a single statement, there is no interleaving window *within* the insert. Re-claim by the current holder is an idempotent upsert (refresh `last_seen`); release is idempotent (delete-if-exists). Expiry is swept lazily on the next read (`DELETE WHERE expires_at < now`). The four behaviors are exactly properties I2–I4. □ □

Pitfall. Theorem 6.1 is sound for the *fresh-contention* case. There is a subtler hazard the dossier elided: the lock path runs `releaseExpired` (a `DELETE`) and *then* `acquire` (the `INSERT`) as two separate statements, not one transaction (`lib/locks.ts`). On the single daemon connection this is serialized by the connection itself; *across* connections (the scenario the single-writer prose sometimes invokes), the default `DEFERRED` transaction opens a window for `SQLITE_BUSY` mid-sequence rather than clean serialization. The fix is `BEGIN IMMEDIATE` (or wrapping acquire in a transaction); it is *absent* from that path today. So Theorem 6.1 holds because all writes route through the one daemon connection (Def. 4.1), *not* because the expire-then-acquire sequence is itself atomic. State which story you mean; do not tell both.

6.3 The missing state: fairness

The conspicuous gap in Figure 5 is the absence of a QUEUED state. Locks have a TTL but *no* wait-list and *no* fairness: acquisition is racy first-come, so a fast agent churning a hot file can starve a slow agent indefinitely. There is no priority, no FIFO wait-list, no anti-starvation guarantee. This is open problem OP-1, and it has teeth: adding bounded-wait may force a real scheduler into the kernel, which would push L0 toward L1.

Exercises.

(check) Why does releasing a lock you do not hold return success rather than an error? Which property (I2–I4) is this, and what would break if it raised?

(trace) Two agents call `acquire` on the same fresh key at the same instant. Walk the `PRIMARY KEY` insert for both and show exactly where the loser learns the holder's identity.

(open, ★ OP-1) Can L0 add a bounded-wait fairness guarantee to claims/locks while keeping the single-writer, no-background-scheduler simplicity? Is a SQLite-backed FIFO wait-list with TTL'd reservations enough, or does fairness force a scheduler (and thus blur the L0/L1 boundary)?

7 The communication organ: the bus, stigmergy, and two delegation chains

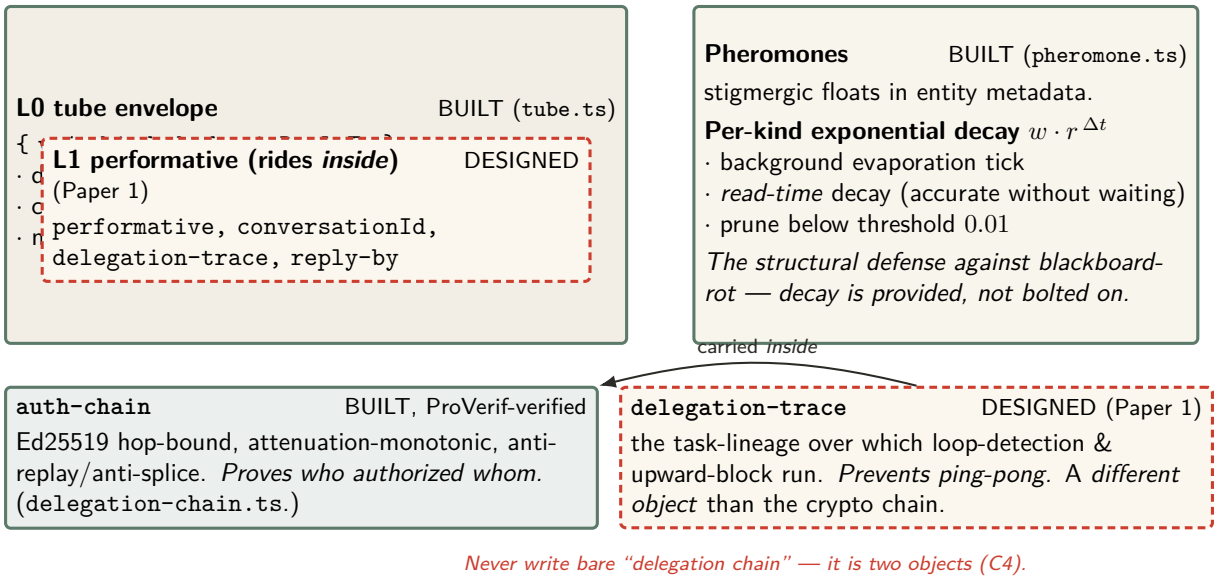


Figure 6: The communication organ. The L0 **tube** envelope — durable, ordered, at-least-once, history-cursored (BUILT) — is the carrier *into which* the L1 typed performative is layered; the kernel ships the envelope, Paper 1 ships the semantics that ride inside it. **Pheromones** implement stigmergic coordination with per-kind exponential decay computed both on a background tick and *at read time*, so the anti-blackboard-rot property is provided by the substrate rather than left to discipline. Finally, the figure fixes the volume's most dangerous naming collision (C4): `auth-chain` is the cryptographic, ProVerif-verified Ed25519 chain that proves *who authorized whom*; `delegation-trace` is the coordination object that prevents ping-pong — a different thing entirely, carried *inside* the auth-chain.

The communication organ (Figure 6) is where L0 stops being a lockbox and becomes a substrate for conversation. It is **BUILT**; the *semantics* that ride on top of it (typed performatives, protocol FSMs) are Paper 1's L1 chapter. Three pieces matter here.

7.1 Tube: the carrier envelope

Tube (`lib/tube.ts`) is a durable, ordered, *at-least-once* pub/sub bus over a `messages` table, with a versioned envelope (`{ v:1, kind, body, inReplyTo }`), a history cursor, and message TTL. The L1 typed performative (`performative`, `conversationId`, `delegation-trace`, `reply-by`) is layered *inside* this envelope — the kernel ships the carrier, Paper 1 ships what it carries.

Key idea. At-least-once delivery means a *healthy* bus routinely redelivers and reorders. This collides with the volume's loud-fail honesty discipline (ADR-0045): if "every anomaly is loud," benign redelivery drowns the operator in false alarms. The volume's resolution (stitch C8, specified in Paper 1 and assumed here) is that benign transport non-determinism must be *deduped silently*; only genuine protocol-FSM violations surface loudly. That requires an idempotency key in the envelope — currently absent — and is the cheapest high-leverage L1 fix. The kernel provides the bus; making every performative effect idempotent is L1's debt, flagged here so the reader does not mistake silent dedup for dishonesty.

7.2 Pheromones: decay as a provided guarantee

Pheromones (`lib/pheromone.ts`) are stigmergic floats (Grassé [16]; the Linda tuple-space lineage of Gelernter [17] is the sibling primitive in `lib/tuples.ts`) stored in entity metadata, with per-kind exponential decay $w \cdot r^{\Delta t}$ computed on a background evaporation tick *and* at read time (so a reader sees the correct decayed value without waiting for the tick), pruning below a threshold of 0.01. The decay is the structural defense against "blackboard-rot": stale coordination signals evaporate on their own. The calibration of the decay rate is genuinely open (OP-8): too slow and signals rot, too fast and they evaporate before they coordinate.

7.3 Two delegation chains, one dangerous name

The volume's most dangerous naming collision (stitch C4) lives in this organ. "Delegation chain" names *two different objects*:

- `auth-chain` — the *cryptographic* object (`lib/delegation-chain.ts`, **BUILT**, ProVerif-verified): an Ed25519 hop-bound, attenuation-monotonic, anti-replay/anti-splice chain that proves *who authorized whom*. This is what Paper 4's cross-harbor capability transfer rides.
- `delegation-trace` — the *coordination* object (**DESIGNED**, Paper 1): the task-lineage over which loop-detection and upward-block run, preventing ping-pong.

These are distinct objects, and the relationship is that the `delegation-trace` is carried *inside* the `auth-chain` — loop-detection runs over a lineage whose authenticity the crypto chain guarantees. We never again write bare "delegation chain."

Pitfall. The "trace inside chain" relationship is a slogan until the task-shape field is in the *signed* payload at each hop — and that collides with the project's no-keyword-NLP rule, because loop-detection needs a semantic task-shape equivalence (embeddings or a structural hash), which is not deterministically signable. This tension is a real open problem; we flag it (it is formalized in Paper 1 and as OP-2's cousin) rather than assert it solved.

7.4 A second transport

The kernel ships *two* transports, not one: HTTP-over-Unix-socket *and* a binary IPC framing over a Unix domain socket (`lib/ipc-*.ts`) with peer-credential authentication and backpressure/draining — the high-frequency path for tight-loop coordination. We note in §13 that the peer-credential check is a software handshake, not kernel-enforced `SO_PEERCRED` (a real trust-boundary caveat).

Exercises.

(check) Why does read-time pheromone decay make the system more honest than a background tick alone? Give a scenario where tick-only decay would report a stale signal as fresh.

(trace) A `request` performative is sent over tube and redelivered once by the at-least-once bus. Trace what the operator should see under the C8 resolution, and what an envelope idempotency key would change.

(open, ★ OP-8) What per-kind pheromone decay rate keeps stigmergic signals neither rotten nor prematurely evaporated? Frame this as an empirical-systems measurement, not a constant to guess.

8 The obligation & enforcement organ: the sovereign's two arms

This organ is the deontic core, and it is genuinely two distinct mechanisms (Jones & Sergot [5]): *prohibition* (the Arbiter) and *obligation* (commitments). Getting the distinction right — and being honest about how strong each arm actually is — is the security heart of the paper.

Prohibition
"you must not"

Two mechanisms, by rule:

Regimented (pre-commit, truly unreachable):
double-claim, PID-squat, prod-DB-in-test, `attest` refuse-to-serve.
Credit: PK constraint / boot gate.

Enforced (post-commit, detected & compensated):
`HEARTBEAT_FRESHNESS`, `NOTE_MONOTONICITY`, `CAP_ESCALATION`, `LOCK_OWNER_VALID`.
The Arbiter, a log subscriber.

Obligation
"you ought"

Enforced by durable, *violable* commitments (Cohen & Levesque persistent goals):

Law 1 — `due_at` is daemon-derived, *never* agent-supplied (Goodhart-resistance, I5).

Law 2 — `close()` refuses 'done' without a non-empty `closed_by_oracle_ref` (a released claim, a merged SHA, a passing test, a satisfied Arbiter sub-check). Free text cannot close (I6).

State machine: `open` → `done` | `abandoned` | `superseded`.

Permission
"you may"

A recorded **capability** attached to an actor.

The split that must not collapse (C9):

capability = *ability* (what the jail makes possible).

permission = *normative status* (what the deontic layer allows).

"Able but forbidden" must stay representable — e.g. the `-allow-main-worktree` flag that *exists* (capable) but *must not be used* (forbidden).

The deontic theorem of the kernel (Jones & Sergot 1993): *prohibition* is regimented *or* detected; *obligation* is enforced by oracle-bound commitments; *permission* is a recorded capability — and L1's deontic binding (ADR-0047) is a direct lift of this split.

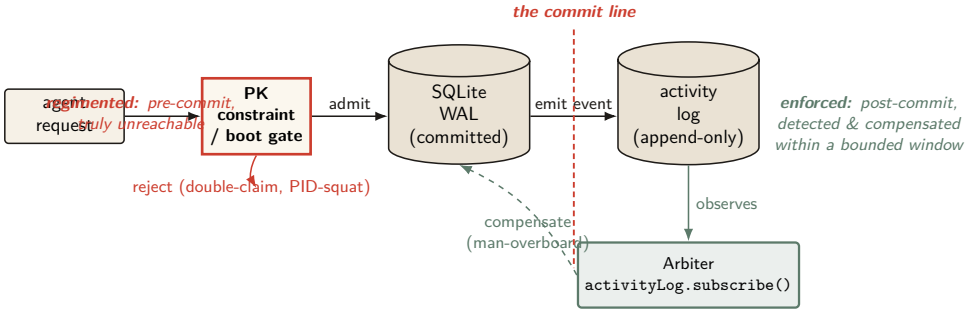
Figure 7: The kernel's deontic split, the formal core of its obligation/enforcement organ. Following Jones & Sergot, the daemon realizes three deontic modalities with three distinct mechanisms. *Prohibition* is handled two ways depending on the rule (the C1 correction): a small regimented set is rejected pre-commit by a `PRIMARY KEY` constraint or a fail-closed boot gate (truly unreachable); everything the Arbiter *subscribes* to is detected post-commit and compensated. *Obligation* is enforced by durable, violable commitments whose deadline the agent cannot set (Law 1) and whose closure demands external evidence (Law 2). *Permission* is a recorded capability — and, crucially, capability (ability) is kept distinct from permission (normative status) so that "able but forbidden" remains expressible, the very state the project's own bypass-flag discipline depends on.

8.1 The deontic split

Definition 8.1 (The kernel's deontic theorem)**.** The kernel realizes three deontic modalities with three mechanisms: *prohibition* is regimented (pre-commit) or enforced (post-commit, detected); *obligation* is enforced by oracle-bound commitments; *permission* is a recorded capability. L1's deontic binding (ADR-0047) is a direct lift of this split.

We keep capability (*ability* — what the jail makes possible) distinct from permission (*normative status* — what the deontic layer allows), per stitch C9, so that "able but forbidden" stays expressible. The canonical example is the project's own `-allow-main-worktree` flag: it *exists* (an agent is *capable* of invoking it) but *must not be used* (it is *forbidden*). Collapsing capability into permission makes that state inexpressible, which is exactly the bug a guardrail-bypass discipline cannot afford.

8.2 The Arbiter is a detector, not a regimenter



Reference-monitor properties (Anderson 1972 / Lampson): (1) *always invoked* (complete mediation) — **satisfied for the PK/boot gate** by the single SQLite file lock; **NOT** satisfied by the Arbiter, which runs after commit. (2) *tamper-evident* — the Merkle-chained audit log (re-scoped to L3, §13.3). (3) *small enough to verify* — one writer, one file, a deliberately shrunk TCB.

Figure 8: The kernel as a reference monitor — and the honest correction that the Harbor Volume's stitch report (C1) forces. Anderson's reference monitor must be *always invoked on the request path*, *before* the operation commits. The **PK constraint / boot gate** (cinnabar) meets this for double-claim, PID-squatting, and the prod-DB-in-test guard: it rejects *before* the WAL write. The **Arbiter** (patina) does not: `arbiter.ts:328` is an `activityLog.subscribe(...)` callback that fires *after* the event is already durable. By Schneider's theorem (§8) a monitor downstream of the commit line enforces no safety property — it can only *detect and compensate*. We therefore reserve "physically unreachable" for the regimented set and say "detected within a bounded window, then halted/compensated" everywhere else.

Here is the correction the whole volume must adopt (stitch C1). The most-repeated claim about the Arbiter — that it "makes forbidden coordination states physically unreachable" — is false as stated, and falsely inherited by all four papers. The code is decisive: `arbiter.ts:328` is `activityLog.subscribe((entry) => {...})`. The Arbiter is a *subscriber to an append-only event stream that has already committed*. By the time it observes a `security.violation`, the offending write is durable in the WAL. Even in strict mode, its strongest response is a *compensating* "man-overboard" salvage — an undo, not a prevention.

Theorem 8.1 (Schneider bound on the Arbiter)**.** A monitor that observes the activity log strictly after commit enforces no safety property; it can only detect and compensate. Therefore every Arbiter *subscription* rule (`HEARTBEAT_FRESHNESS`, `NOTE_MONOTONICITY`, `CAP_ESCALATION`, `LOCK_OWNER_VALID`, ...) is, architecturally, detect-and-compensate — not regimentation.

Proof sketch. Schneider's theory of enforceable security policies via execution monitors [4] establishes that an execution monitor can enforce a property only by *truncating the execution before the bad state is entered*. A monitor invoked strictly after the state transition has committed cannot truncate before it; the bad state already exists. Hence it can detect the violation and trigger a compensating transition, but it cannot have prevented the state. Subscription to a post-commit log places every subscribed rule in this regime. □ □

Key idea. The real, inline regimentation is credited to the right mechanism: the `PRIMARY KEY` constraint (double-claim, PID-squat-as-PK) and the fail-closed boot gate (prod-DB-in-test, `attest` refuse-to-serve). Those reject *before* the commit line and are *truly* unreachable. The Arbiter notices everything else *afterward* and compensates. So: "physically unreachable" for the regimented set; "detected within a bounded window, then halted/compensated" for the rest. Crediting double-claim prevention to the Arbiter *double-counts* — the PK already did it; the Arbiter just logs it.

The Arbiter is honest about its own coverage, which is a genuine security property: `ArbiterStatus` reports each rule as `enforced`, `degraded`, or `stubbed`, and one rule (`CAP_ESCALATION`) is backed by a Rust FFI core. We note in §13 that the FFI enforcement and its temp-file materialization are the weakest-evidenced claim in the organ and carry a time-of-check/time-of-use surface.

8.3 Commitments: obligation with oracle-bound closure

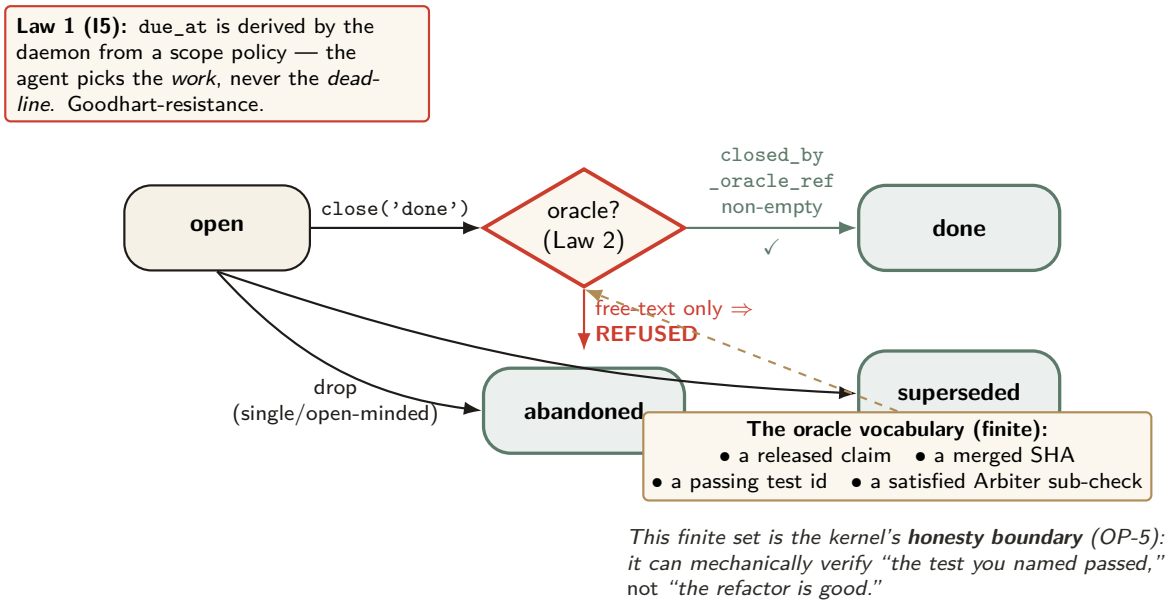


Figure 9: The commitment lifecycle with its oracle-bound closure gate. A commitment is a durable, *violable* promise (Cohen & Levesque's persistent goal): it may end `done`, `abandoned`, or `superseded`. The load-bearing gate is Law 2: a transition to `done` is *refused* unless the agent supplies a non-empty `closed_by_oracle_ref` drawn from a finite vocabulary — a released claim, a merged SHA, a passing test id, or a satisfied Arbiter sub-check. Free-text notes cannot close a promise. That finite vocabulary is precisely the kernel's honesty boundary: it can verify "the test you named passed," never "the work is good" (open problem OP-5). Law 1 (daemon-derived deadlines) closes the complementary Goodhart hole — the agent chooses the work, the daemon chooses when it is due.

Commitments (`lib/commitments.ts`, `lib/obligation-monitor.ts`) are durable, *violable* promises — Cohen & Levesque's "intention as persistent goal" [14], with single-minded and open-minded drop strategies. Two of five hardening laws are shipped, and they are the load-bearing ones (Figure 9).

Property 8.1 (Goodhart-resistance, Law 1)**.** A commitment's `due_at` is derived by the daemon from a scope policy and is *never* agent-supplied. The agent picks the work; the daemon picks the deadline. (Invariant I5.)

Property 8.2 (Oracle-bound closure, Law 2)**.** `close('done')` is refused unless the agent supplies a non-empty `closed_by_oracle_ref` drawn from a finite vocabulary: a released claim, a merged SHA, a passing test id, or a satisfied Arbiter sub-check. Free-text notes cannot close a commitment. (Invariant I6.)

Key idea. That finite oracle vocabulary *is* the kernel's honesty boundary. It can mechanically verify "the test you named passed," never "the refactor is good." Naming the oracle set is naming the limit of what L0 can verify — and open problem OP-5 asks which real "done" conditions it fails to capture, and whether the set can be extended without re-opening the Goodhart hole that free-text closure would.

Exercises.

(check) For each Arbiter rule in Theorem 8.1, say whether it *could in principle* be moved to a pre-commit hook (a `BEFORE INSERT` trigger). Which genuinely cannot, and why? (Hint: `HEARTBEAT_FRESHNESS` and Chandra–Toueg [12].)

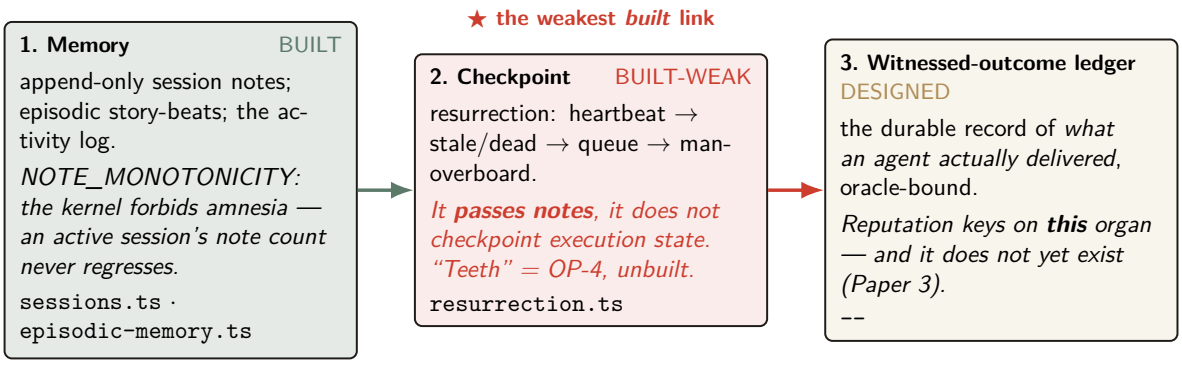
(trace) An agent calls `close('done', oracle_ref="")`. Walk the Law-2 gate and show exactly where and why the transition is refused.

(open, ★ OP-2) Formalize the regimentation-vs-enforcement boundary: which of the Arbiter's rules are truly regimentable (rejectable at insert) versus only enforceable (detectable after)? A clean theorem here is the deontic heart of the layer.

(open, ★ OP-5) Extend the Law-2 oracle vocabulary to capture a "done" condition it currently misses, without re-admitting free text. State your new oracle and prove it is Goodhart-resistant.

9 The continuity organ: memory, checkpoint, and the foundation of the economy

The through-line of the whole volume — memory → checkpoint → continuity → person → reputation → market — *bottoms out here.* If L0's continuity is weak, L3 is built on sand. This is the section Paper 3 and Paper 4 lean on hardest, so it carries the volume's canonical statement of its own softest link.



The canonical sentence (stated identically in Papers 2 and 3): The economy rests on three continuity organs — memory (BUILT), checkpoint (BUILT-WEAK: notes, not execution state), and the witnessed-outcome ledger (DESIGNED) — and reputation keys on the third, which does not yet exist; the checkpoint organ is the weakest *built* link, and "resurrection with teeth" (a real execution-state checkpoint, OP-4) is the literal foundation of L3.

Figure 10: The three continuity organs the entire economy stands on. The through-line of the volume — memory → checkpoint → continuity → person → reputation → market — bottoms out *here*, in the kernel. **Memory** is BUILT and the kernel even forbids amnesia (the Arbiter's `NOTE_MONOTONICITY` rule). **Checkpoint** is BUILT-WEAK: resurrection passes the durable handoff *notes* but does not capture execution state, so it is the weakest built link in the chain. **The witnessed-outcome ledger**, on which reputation actually keys, is DESIGNED only (Paper 3). This figure states the volume's canonical C7 sentence, which appears identically in Papers 2 and 3 so the reader meets one "the foundation is soft here" claim, not three.

9.1 Three organs, one weak link

Figure 10 names the three continuity organs and states the canonical C7 sentence verbatim. We repeat it in prose because the volume requires it identically in Papers 2 and 3:

> *The economy rests on three continuity organs — memory (BUILT), checkpoint (BUILT-WEAK: notes, not execution state), and the witnessed-outcome ledger (DESIGNED) — and reputation keys on the third, which does not yet exist; the checkpoint organ is the weakest* built *link, and "resurrection with teeth" (a real execution-state checkpoint) is the literal foundation of L3.*

Memory (`lib/sessions.ts`, the largest module, plus `lib/episodic-memory.ts` and the activity log) is **BUILT**, and the kernel even forbids amnesia: the Arbiter's `NOTE_MONOTONICITY` rule guards that an active session's durable note count never regresses (detect-and-compensate, per Theorem 8.1, but real).

Checkpoint (`lib/resurrection.ts`) is **BUILT-WEAK**, and the weakness is precise: resurrection runs heartbeat-watchdog → stale/dead detection → resurrection queue → man-overboard salvage, and it *passes notes*; it does not checkpoint execution state. "Resurrection with teeth" — the gap between passing notes and restoring work — is open problem OP-4, the most important unbuilt thing at L0 because it is the literal foundation of the economy.

The witnessed-outcome ledger, the durable record of what an agent actually delivered, is the organ reputation keys on. It is **DESIGNED** only; Paper 3 builds it. OP-4 (here) and the outcome-ledger gap (Paper 3) are the *same* finding viewed from two ends.

9.2 The activity log and tamper-evidence

The append-only activity log (`lib/activity.ts`) is the kernel's audit organ — the stream the Arbiter subscribes to and the thing that makes "what actually happened" answerable. Hash-chained tamper-evidence over it (`lib/merkle-chain.ts`, the Haber–Stornetta [20] pattern that predates blockchain) exists but is re-scoped — see §13.3.

Exercises.

(check) Why is "resurrection passes notes" fundamentally weaker than "resurrection restores work" for an LLM agent that died mid-edit? Name one thing notes preserve and one thing they cannot.

(trace) Follow `NOTE_MONOTONICITY` from an agent appending a note to the Arbiter detecting a regression. At which point is the violation already durable (per Theorem 8.1)?

(open, ★ OP-4) What is the minimal durable artifact that turns "resurrection passes notes" into "resurrection restores work"? Is it a content-addressed snapshot of {working-tree diff + open claims + commitment set + last-N transcript turns}? What is recoverable versus fundamentally lost when an LLM agent dies mid-thought?

10 The self-attestation organ: honest green

ADR-0045 (loud-fail invariants and honest attestation) lives at L0, because the kernel is the one component that can honestly report its own integrity: it is always-on and owns the only writable truth. `pd attest` (`lib/attest.ts`, `lib/attest-invariants.ts`) evaluates an invariant registry where each invariant returns `PASS`, `FAIL`, `SKIPPED(reason)`, or `UNKNOWN`. The report is *scoped and conjunctive*: "all good" is printed only when every checked invariant passes, and the report always enumerates what was *not* checkable.

Property 10.1 (Honest self-report, I10)**.** `pd attest` reports "all good" only if every checked CRITICAL invariant passed, and it always lists the unchecked set. There are three triggers: a boot regimentation gate (refuse-to-serve on CRITICAL fail), a CLI pre-flight (loud refusal naming the fix), and a continuous watchdog (a PASS→FAIL transition emits a pheromone and a notification). **BUILT**.

This is the ADR-0045 honest-label discipline turned on the kernel itself, and it is the right posture for a trusted computing base: when integrity is unknown, deny. The drift/liveness guards (`lib/binary-drift-detector.ts`, `lib/cli-liveness.ts`, `lib/git-origin-check.ts`) exist precisely because a second copy of the daemon *can* run — the homebrew install lagging the repo is a live, recurring hazard — and the kernel must know which copy of itself is the real one.

11 The invariants, stated as theorems

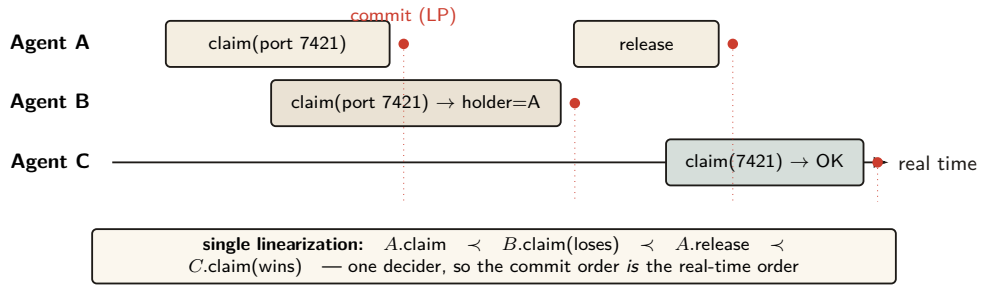
A completionist treatment names the kernel's properties as provable claims, each with its mechanism and honest label. Table 3 is the formal spine of the layer; the honest-label column is the ADR-0045 discipline turned on the theorems themselves.

Table 3: The kernel invariants. I1 is split by fault class (I1a/I1b) per §5; I7–I9 carry the regimentation/detection correction per §8.

#	Invariant (theorem)	Mechanism	Honest label
I1a	Process-crash durability	WAL + OS page cache	**BUILT**
I1b	Power-loss durability	(needs `synchronous=FULL`)	**NOT GUARANTEED**
I2	Mutual exclusion (1 holder/key)	PK constraint + `busy_timeout`	**BUILT**
I3	Idempotence of re-claim/re-release	upsert / delete-if-exists	**BUILT**
I4	Liveness reclamation (TTL sweep)	lazy expiry on read + ticks	**BUILT**
I5	Goodhart-resistance (Law 1)	daemon-derived `due_at`	**BUILT**
I6	Oracle-bound closure (Law 2)	`closed_by_oracle_ref`	**BUILT**
I7	No-amnesia (note monotonicity)	Arbiter `NOTE_MONOTONICITY`	**BUILT-WEAK** (detected, not regimented)
I8	Forbidden-state handling	PK/boot (regimented) + Arbiter (detected)	**BUILT-WEAK** (mostly detect-and-compensate)
I9	Tamper-evidence of the audit log	Merkle chain	**BUILT-WEAK** (re-scoped L3, §13.3)
I10	Honest self-report	`pd attest` scoped report	**BUILT**
I11	Runtime parity (bun ≡ better-sqlite3)	sqlite-runtime shim	**BUILT-WEAK** (OP-3)
I12	Non-forgeable identity	ADR-0040	**DESIGNED** (substrate present, gating absent)

11.1 The consistency-model theorem

The dossier never said “serializable” or “linearizable.” It should have: those words are the formal payoff of the single-writer choice, and they are what make L1’s typed ownership trustworthy.



Theorem (consistency model). Under one writer over a single SQLite database in WAL mode, every transaction is **serializable**; and because the daemon is the sole decider, the externally observable history of claim/lock operations is **linearizable** — each operation appears to take effect atomically at its commit (its *linearization point*), consistent with real-time order. This is the formal payoff of the single-writer choice, and it is what makes L1's typed ownership trustworthy without any agreement protocol.

Figure 11: The consistency-model theorem — the formal property the single-writer choice buys, left implicit in the dossier and made explicit here. Because all mutations serialize through one writer over one WAL file, transactions are **serializable**; because that writer is the sole decider, the observable history of claims and locks is **linearizable**: every operation has a single linearization point at its commit, and the commit order coincides with real time. The trace shows A winning a port, B losing cleanly to the holder, A releasing, and C then winning — one unambiguous order with no agreement protocol in sight. This linearizability is exactly what lets L1 treat "who holds this" as ground truth.

Theorem 11.1 (Consistency model)**.** Under one writer over a single SQLite database in WAL mode, every transaction is **serializable**. Because the daemon is the sole decider, the externally observable history of claim/lock operations is **linearizable**: each operation appears to take effect atomically at its commit (its linearization point), consistent with real-time order.

Proof sketch. SQLite under a single writer provides serializable isolation: there is one stream of write transactions, totally ordered by commit, and readers see a consistent snapshot. For linearizability (Herlihy & Wing [10]) we need a single point per operation, between its invocation and response, at which it appears to take effect, with these points consistent with real time. The commit of each claim/lock operation is exactly such a point: it is atomic (Theorem 6.1), it lies between the client's request and the daemon's response, and because the sole daemon connection commits operations in a total order that respects the order in which it received them, the linearization order coincides with real time. Hence the observable claim/lock history is linearizable. □ □

Key idea. This is why the kernel needs no agreement protocol. Linearizability is the strongest single-object consistency guarantee, and the single-writer architecture gives it *for free* — not despite refusing consensus, but *because* it refused consensus. One decider, one order, one truth.

11.2 The dual-runtime hazard, and what would make I11 a theorem

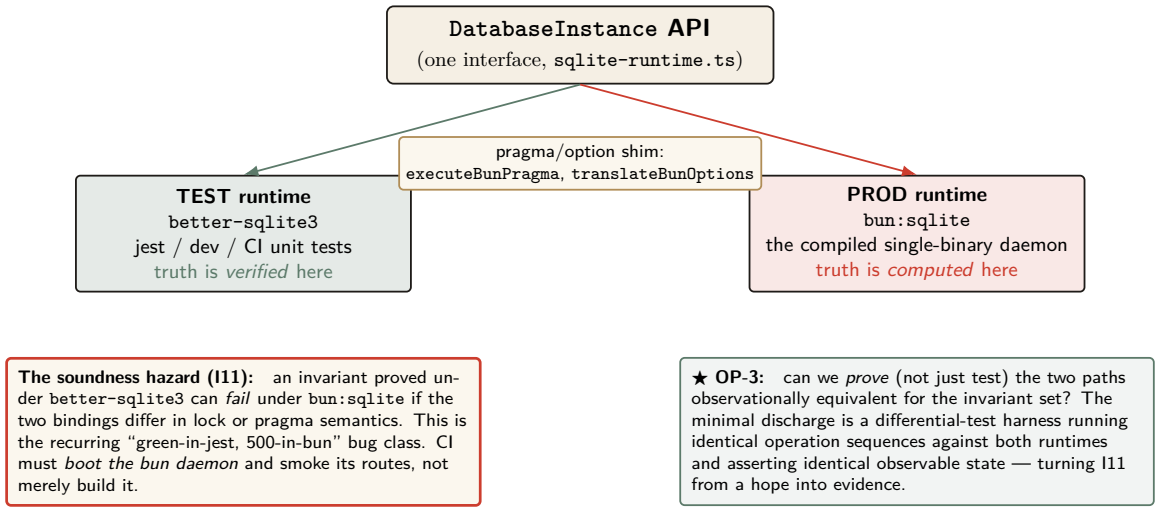


Figure 12: The dual-runtime substrate, a genuine kernel property the seeds treat as an implementation detail. One `DatabaseInstance` interface is satisfied by *two* SQLite bindings: `better-sqlite3` under jest (where invariants are *verified*) and `bun:sqlite` in the compiled daemon (where truth is actually *computed*). A pragma/option shim bridges them, but the bridge is not a proof: an invariant green under test can be 500 in production if the bindings diverge in lock or pragma semantics — the recurring "green-in-jest, 500-in-bun" failure. Invariant I11 (runtime parity) is therefore BUILT-WEAK, and open problem OP-3 asks for the differential-test harness that would make it a theorem.

Invariant I11 (runtime parity) is the one place the formal spine is held together by a shim rather than a proof. Truth is *computed* under `bun:sqlite` (the compiled single-binary daemon) but *verified* under `better-sqlite3` (jest); a pragma/option shim (`lib/sqlite-runtime.ts`) bridges them (Figure 12). An invariant green under test can be a 500 in production if the bindings diverge in lock or pragma semantics — the recurring "green-in-jest, 500-in-bun" failure. CI must *boot the bun daemon* and smoke its routes, not merely build it. Open problem OP-3 asks for the differential-test harness that would turn I11 from a hope into evidence.

Exercises.

(check) Theorem 11.1 gives linearizability "for free." What does "free" cost? (Hint: `busy_timeout` and throughput under contention — the single writer is also a scalability ceiling.)

(trace) Construct a three-operation claim history and give its unique linearization. Then show why no second linearization is consistent with real time.

(open, ★ OP-3) Specify the minimal differential-test suite that would make I11 a theorem: what operation sequences, what observable-state assertions, run against both runtimes?

12 Cross-organ atomicity: a buildable defect, not a frontier

"Claim this port *and* open this session *and* record this commitment" is three writes. Nothing at the kernel's API boundary wraps them in one SQLite transaction, so partial-failure semantics across organs are undefined. The dossier labeled this **VISION**. That is wrong, and the correction matters because it changes the engineering posture from "research" to "do it."

Key idea. This is the Saga problem (Garcia-Molina & Salem [15]) — but with a *cheap local fix*. These are all local writes to one file; `db.transaction()` already exists in the codebase (it is used in `bonds.ts` and `symbol-index.ts`). Wrapping a multi-organ operation in one local transaction (or `BEGIN IMMEDIATE`) gives all-or-nothing for free. Calling it **VISION** understates how cheap the correct answer is. It is a **buildable defect**, re-labeled here from **VISION** to "buildable now."

The reason it is not yet built is simply that no one wrapped the multi-organ API endpoints; the primitive is present. This is the kind of finding the honest-label discipline exists to surface: a gap mis-labeled as a frontier hides a one-line fix behind the word "VISION."

Exercises.

(check) Give a concrete partial-failure: port claimed, session opened, then the commitment write fails. What inconsistent state is left, and which agent sees it?

(open, ★ C5) Sketch the API change that wraps the three-organ "begin" operation in one `db.transaction()`. What is the compensating action if you instead chose Sagas, and why is the transaction strictly simpler here?

13 Threat model and the limits of mediation

A kernel paper lives or dies on its threat model. The kernel deliberately shrinks its trusted computing base (one writer, one file), which is good security hygiene — but the shrinking pushes several adversaries explicitly out of scope, and the honest move is to draw the boundary, not gesture at it.

Table 4: The L0 threat model. The same-user adversary is *out of scope* by design; several "security" organs only become load-bearing against the cross-machine adversary, which is L3 (Paper 4).

Adversary	In/out of scope	What defends (or does not)
Buggy cooperating agent	**in**	Claims, commitments, Arbiter detection, oracle-bound closure. The primary design target.
Misbehaving agent (ignores advisory claims)	partial	`session_files` claims are *advisory coordination*, not OS-enforced isolation. A malicious agent can write the file anyway (no Landlock/unveil/Seatbelt).
Same-user process (reads/edits the DB)	**out**	The DB is `0600`; the signing key is mid-migration to the OS keychain. Merkle tamper-evidence defends against *nobody* under this exclusion (§13.3).
Different-uid process	**in (partial)**	`0600` perms; but IPC peer-cred is a software handshake, not `SO_PEERCRED` — a real authentication caveat.
Cross-machine / cross-operator	**out (it is L3)**	Needs non-forgeable identity (ADR-0040, **DESIGNED**) and cross-operator attestation (Paper 4).

13.1 Advisory claims are not sandboxing

The sharpest category error to avoid: `session_files` claims are *advisory coordination locks between cooperating agents*, not OS-enforced filesystem isolation. There is no seccomp-bpf, no `pledge`/`unveil`, no macOS Seatbelt, no Landlock. A malicious agent ignores a claim row and writes the file. The kernel provides *advisory* coordination; real OS sandboxing of agent processes is out of scope at L0 (it is the L1 "jail," **DESIGNED** in Paper 1, and would need to be grounded in a real OS primitive). We say so plainly rather than let advisory claims be read as isolation.

13.2 Self-asserted identity bounds every other invariant

Today `owner_actor_id` is whatever the session claims (I12 is **DESIGNED**). This silently bounds the security weight of I7, I8, and I9: every rule that keys on identity (`LOCK_OWNER_VALID`, `CAP_ESCALATION`, `NOTE_MONOTONICITY`, owner-scoped release) is unforgeable only up to impersonation. A malicious agent simply claims to be another actor. The Rust-FFI `CAP_ESCALATION` enforcement is theater if the capability set it checks hangs off a forgeable identity. This dependency — I12 gates the soundness of I7/I8/I9 — is a hard precondition, stated here so the reader does not over-trust the enforcement organ.

13.3 Merkle tamper-evidence, re-scoped to L3 provisioning

The Merkle chain (I9) is built, but under the *stated* threat model it defends against nobody: its only adversary is someone who can edit the audit log, which is exactly the same-user process the model excludes (OP-9). Tamper-evidence that nothing on the read path verifies is a data structure, not a control. We therefore re-scope I9 as **L3 provisioning**: it matters only against cross-machine sync or a non-same-user tamperer. Its load-bearing treatment moves to Paper 4; here it is a one-line pointer: *provisioned at L0, activated at L3.*

Exercises.

(check) The Merkle chain defends against nobody in scope. Name the *out-of-scope* adversary it does defend against, and the layer (L0/L3) where that adversary becomes real.
(trace) An agent forges `owner_actor_id` to release another agent's lock. Walk `LOCK_OWNER_VALID` and show exactly where I12's absence lets the forgery through.
(open, ★ OP-9) What is the minimal hardening that closes the same-machine adversary without a TPM dependency — OS keychain for the signing key, sealed memory? Where does each stop?

14 Adjacency contract: what the kernel assumes and provides

The kernel's coherence with the layers around it is a contract: what it assumes from the machine below, what it provides to the protocol above, and — just as important — what it explicitly does *not* provide so higher layers do not over-assume.

14.1 What L0 assumes from the machine

- A POSIX filesystem with working advisory locking (`flock`/`fcntl`). **This is a correctness precondition for I2**: networked filesystems break advisory locks, and a broken lock lets two writers both win — destroying mutual exclusion, not just performance.
- A local wall clock (`Date.now()`). Single-machine, so no shared-clock assumption is needed — but note that `Date.now()` is *not monotonic* (it steps on NTP adjustments), an intra-node

hazard for every `expires_at < now` sweep that the Lamport [13] inter-node citation does not cover.

- Unix domain sockets with peer-credential auth (the IPC and HTTP transports).
- An OS keychain for signing keys; `chmod` honored on the DB path.
- A supervisor (launchd) to keep the daemon always-on and to resurrect *the daemon itself* — L0 makes other things always-on but cannot make *itself* always-on; that is delegated downward.

14.2 What L0 provides to L1 (the kernel API contract)

- **Atomic, idempotent, TTL'd claims** (I2–I4) over ports, file-regions, symbols, and named locks — L1's typed ownership reduces to L0 claim release.
- **A durable, versioned, history-cursored pub/sub bus** (tube) + tuple space + decaying pheromones — L1's performative envelope is carried *in* L0's tube envelope; the anti-blackboard-rot property is *provided* by L0 decay.
- **The deontic split as a primitive** (Def. 8.1): prohibition → Arbiter (detect/regiment), obligation → commitment, permission → capability. L1 must not re-implement enforcement.
- **Daemon-owned deadlines and oracle-bound closure** (I5, I6).
- **An append-only audit log** (Merkle-provisioned) the protocol and the operator read.
- **Self-attestation** (I10) — higher layers may *assume the kernel is honest about its own health.*
- **The cryptographic `auth-chain`** — L1's `delegation-trace` rides inside it.
- **Linearizable claim/lock semantics** (Theorem 11.1) — the property that lets L1 treat "who holds this" as ground truth.

14.3 The explicit non-provisions

- L0 does *not* provide non-forgeable identity yet (I12, **DESIGNED**).
- L0 does *not* provide fair/queued exclusion (OP-1), cross-organ transactional atomicity by default (§12), real execution-checkpoint (OP-4), or any cross-machine guarantee (L3 — different theorems, a shared-clock and Byzantine-membership problem L0 deliberately never touches).
- L0 does *not* decide *what to do* (no orchestration). It enforces what *cannot* be done and records what *did* happen — the "building department, not architect" model.

15 Open problems

These nine open problems are the layer's research frontier; they appear as starred exercises throughout and are collected here. OP-4 is co-owned with Paper 3 (it owns the kernel mechanism; Paper 3 owns the personhood-and-reputation argument).

★ OP-1 — Fair exclusion without a scheduler.
Add bounded-wait to claims/locks while keeping the single-writer, no-background-scheduler simplicity. Is a SQLite-backed FIFO wait-list with TTL'd reservations enough?

★ **OP-2 — Regimentation vs. enforcement boundary.**
Formalize which Arbiter rules are truly regimentable (rejected at insert) versus only enforceable (detected after). The deontic heart of the layer (Theorem 8.1).

★ **OP-3 — Cross-runtime soundness as a proof obligation.**
Prove, not test, that the bun:sqlite and better-sqlite3 paths are observationally equivalent for the invariant set (I11).

★ **OP-4 — Checkpoint with teeth.**
The minimal durable artifact that turns "resurrection passes notes" into "resurrection restores work." The literal foundation of the L3 economy.

★ **OP-5 — Oracle completeness.**
Extend the finite Law-2 oracle set without re-admitting free-text (without re-opening the Goodhart hole).

★ **OP-6 — Tamper-evidence on the read path.**
Where should Merkle verification gate? A sampling/checkpoint regime with a provable detection-probability bound (couples to §13.3's L3 re-scope).

★ **OP-7 — Schema evolution without a sequencer.**
Safe renames/backfills/down-migrations preserving "any module, any order."

★ **OP-8 — Pheromone decay calibration.**
The right per-kind decay so stigmergic signals neither rot nor evaporate before they coordinate.

★ **OP-9 — Same-machine adversary.**
Minimal hardening (OS keychain, sealed memory) that closes the same-user process adversary without a TPM.

16 Conclusion: solid where local, provisional where it must grow

The kernel is a single-writer transactional reference monitor over a local SQLite/WAL file, and its two jobs — durable record and mediated exclusion — are real and, mostly, proven. It earns its strongest claims (mutual exclusion, serializable/linearizable consistency, oracle-bound closure) precisely by refusing the distributed-systems problem it superficially resembles: one writer, one file, one decider, no consensus.

Where it stops, it stops honestly. Durability is split: process-crash durable (I1a), *not* guaranteed against power loss (I1b). The Arbiter detects and compensates; it does not regiment, and we credit the real inline regimentation to the PK constraint and boot gate. Cross-organ atomicity is a buildable defect, not a frontier. Merkle tamper-evidence is re-scoped to L3 provisioning, where it actually defends someone. And the two genuinely soft spots — non-forgeable identity (I12) and checkpoint-with-teeth (OP-4) — are exactly the two things the economy (Paper 4) and the person (Paper 3) lean on hardest.

> *The kernel is solid where it is local and provisional exactly where the economy will need it to be cryptographic and continuous. That is not a flaw to hide; it is the layer boundary, stated truthfully.*

Every "single-machine / one-writer / one-clock / self-asserted-identity" simplification that makes this layer clean is precisely the assumption a later layer must pay to relax. The Anchor

Protocol *proper* — cross-harbor capability transfer over the `auth-chain` this kernel ships — is where that payment begins, and it is Paper 4, not here. Build the harbor first. The cryptography earns its keep when the ships sail out to trade.

References

[1] James P. Anderson. Computer Security Technology Planning Study. ESD-TR-73-51, U.S. Air Force Electronic Systems Division, 1972. (The origin of the *reference monitor* concept.)

[2] Butler W. Lampson. Protection. *ACM SIGOPS Operating Systems Review*, 8(1):18–24, 1974.

[3] Jerome H. Saltzer and Michael D. Schroeder. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[4] Fred B. Schneider. Enforceable Security Policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000.

[5] Andrew J. I. Jones and Marek Sergot. On the Characterisation of Law and Computer Systems: The Normative Systems Perspective. In *Deontic Logic in Computer Science*, Wiley, 1993.

[6] Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992. (Atomicity/consistency vs. durability — the ground for I1a/I1b.)

[7] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992.

[8] D. Richard Hipp et al. SQLite Write-Ahead Logging. SQLite Documentation, 2010. `https://www.sqlite.org/wal.html` (`synchronous=NORMAL` "commits might lose durability following a power loss.")

[9] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly, 2017. (Single-leader as the right default; consensus only when forced.)

[10] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990.

[11] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of Distributed Consensus with One Faulty Process. *Journal of the ACM*, 32(2):374–382, 1985.

[12] Tushar Deepak Chandra and Sam Toueg. Unreliable Failure Detectors for Reliable Distributed Systems. *Journal of the ACM*, 43(2):225–267, 1996.

[13] Leslie Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7):558–565, 1978.

[14] Philip R. Cohen and Hector J. Levesque. Intention Is Choice with Commitment. *Artificial Intelligence*, 42(2–3):213–261, 1990.

[15] Hector Garcia-Molina and Kenneth Salem. Sagas. In *ACM SIGMOD International Conference on Management of Data*, 1987.

[16] Pierre-Paul Grassé. La reconstruction du nid et les coordinations interindividuelles…: la théorie de la stigmergie. *Insectes Sociaux*, 6(1):41–80, 1959.

[17] David Gelernter. Generative Communication in Linda. *ACM Transactions on Programming Languages and Systems*, 7(1):80–112, 1985.

[18] Marco Dorigo and Gianni Di Caro. The Ant Colony Optimization Meta-Heuristic. In *New Ideas in Optimization*, McGraw-Hill, 1999.

[19] Ralph C. Merkle. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology — CRYPTO '87*, 1987.

[20] Stuart Haber and W. Scott Stornetta. How to Time-Stamp a Digital Document. *Journal of Cryptology*, 3(2):99–111, 1991.

[21] Jeffrey O. Kephart and David M. Chess. The Vision of Autonomic Computing. *IEEE Computer*, 36(1):41–50, 2003.

[22] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-Speed High-Security Signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012. (Ed25519 — the `auth-chain` primitive.)

[23] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016. (The tool verifying the `auth-chain`; full treatment in Paper 4.)

[24] Michael T. Nygard. *Release It! Design and Deploy Production-Ready Software*. 2nd ed., Pragmatic Bookshelf, 2018. (`busy_timeout` as a missing bulkhead/timeout-budget under contention.)